Parallel and Distributed Systems 

Pr. Olivier Gruber

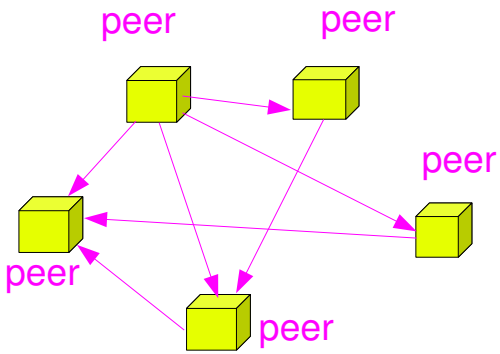
olivier.gruber@univ-grenoble-alpes.fr

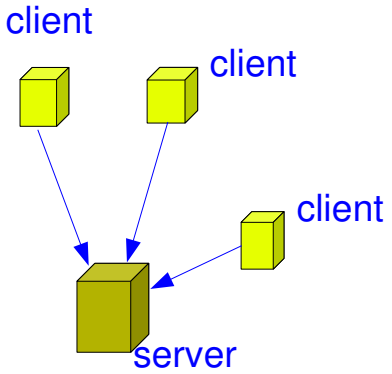
Université Grenoble Alpes

©Pr. Olivier Gruber

Group – Multicast vs Broadcast

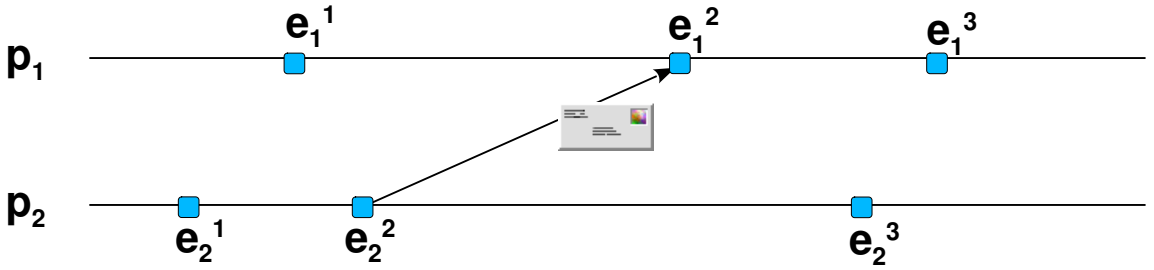
- Group of actors
  - We are discussing groups because ordering is easier
  - A group of actors where each actor knows the others and can communicate with them
- Broadcast – no ordering
  - Usually no ordering is implied
  - Reliable broadcasts provide atomicity but not ordering
- Multicast – two possible orders
  - Causal order
    - Example: chat room (human discussion)
  - Total order
    - Example: locking





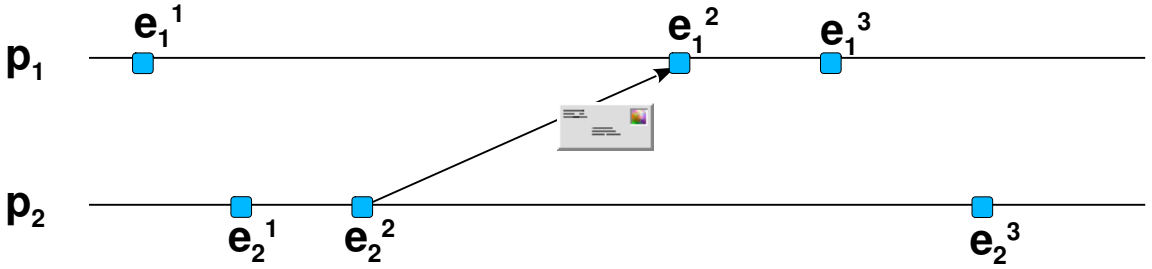
Causal Order

- Example
  - We have
    - $e_1^1 \rightarrow e_1^2 \rightarrow e_1^3$
    - $e_2^1 \rightarrow e_2^2 \rightarrow e_2^3$
    - $e_2^2 \rightarrow e_1^2$
  - Therefore we have
    - $e_2^2 \rightarrow e_1^3$
  - Only a ***partial order***
    - *Comparing any other pairs of events, we have* $e_i^j \parallel e_p^q$
    - *For instance, we neither have* $e_1^1 \rightarrow e_2^1$ *or* $e_2^1 \rightarrow e_1^1$



Causal Order

- Example
  - We have
    - $e_1^1 \rightarrow e_1^2 \rightarrow e_1^3$
    - $e_2^1 \rightarrow e_2^2 \rightarrow e_2^3$
    - $e_2^2 \rightarrow e_1^2$
  - Therefore we have
    - $e_2^2 \rightarrow e_1^3$
  - Only a ***partial order***
    - *Comparing any other pairs of events, we have* $e_i^j \parallel e_p^q$
    - *For instance, we neither have* $e_1^1 \rightarrow e_2^1$ *or* $e_2^1 \rightarrow e_1^1$



Causality Example

- A simple loop:

```
int vals[]={0,1,2,3}
for (int i=1; i < vals.length;i++)
    vals[i] = vals[i] + vals[i-1];
```

This execution is causal…

It means that causes happen before consequences...

Causality Example

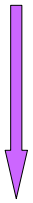
- A simple loop:

```
int vals[]={0,1,2,3}
for (int i=1; i < vals.length;i++)
    vals[i] = vals[i] + vals[i-1];
```

This execution is causal…

"Causes happen before consequences"

Each loop iteration uses a value computed in a previous iteration



```
int vals[]={0,1,2,3}

vals[1] = vals[1] + vals[0];
vals[2] = vals[2] + vals[1];
vals[3] = vals[3] + vals[2];
```

Any sequential execution is causal…

In fact, real life is causal...

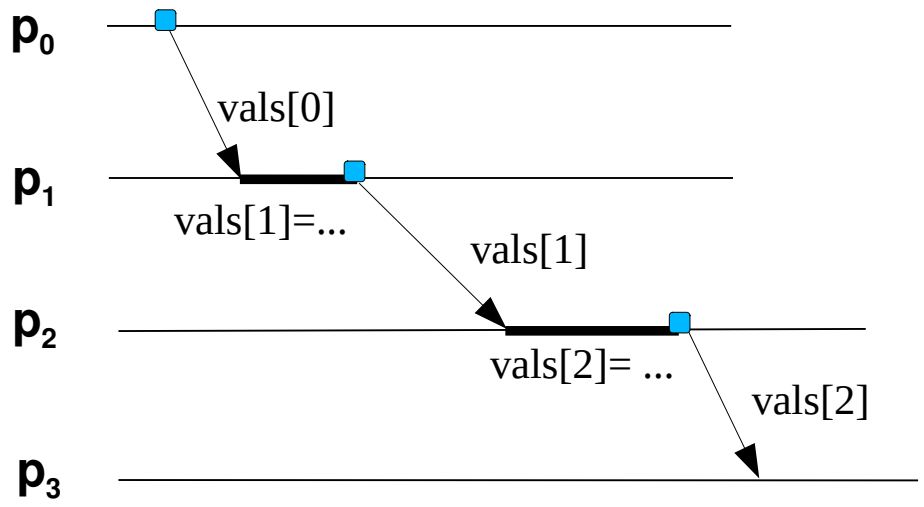
Causality Example

- A simple loop:

```
int vals[]={0,1,2,3}
for (int i=1; i < vals.length;i++)
    vals[i] = vals[i] + vals[i-1];
```

Distributed design:

- Distributed values: vals[i] on processus Pi
- Distributed the computation: send computed values when appropriate



This is a simple design…
This is a correct design…

vals[i] is computed before it is used.

send(m) → send(m') → send(m")

deliver(m) → deliver(m') → deliver(m")

Causality Example

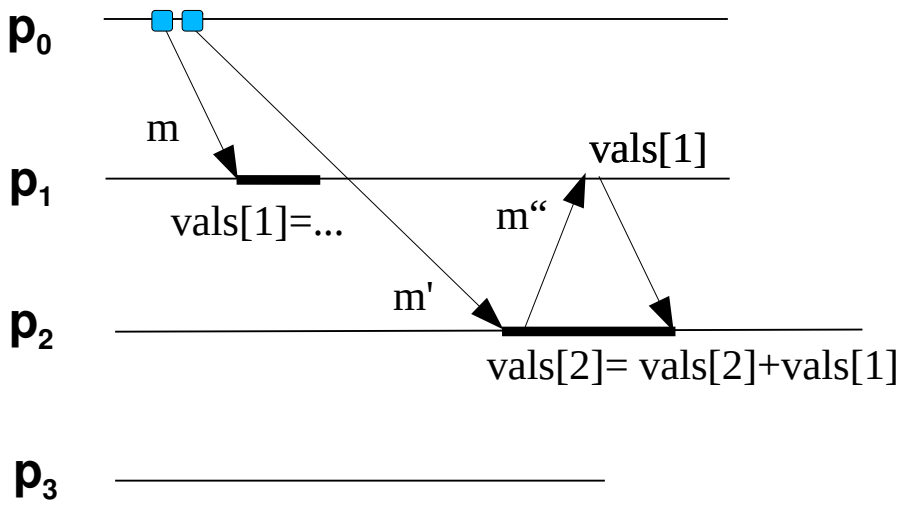
- A simple loop:

```
int vals[]={0,1,2,3}
for (int i=1; i < vals.length;i++)
    vals[i] = vals[i] + vals[i-1];
```

A different design:

- Distributed values: vals[i] on processus Pi
- Loop at P_0 sending asynchronous requests to compute vals[i] at Pi

```
At P0:
  void init(Task tasks[]) {
    for (int i=1; i < tasks.length;i++)
      vals[i] = tasks[i].computeValue();
  }
```



An “optimization”…
going for parallel execution…

Is this still correct?

Causality Example

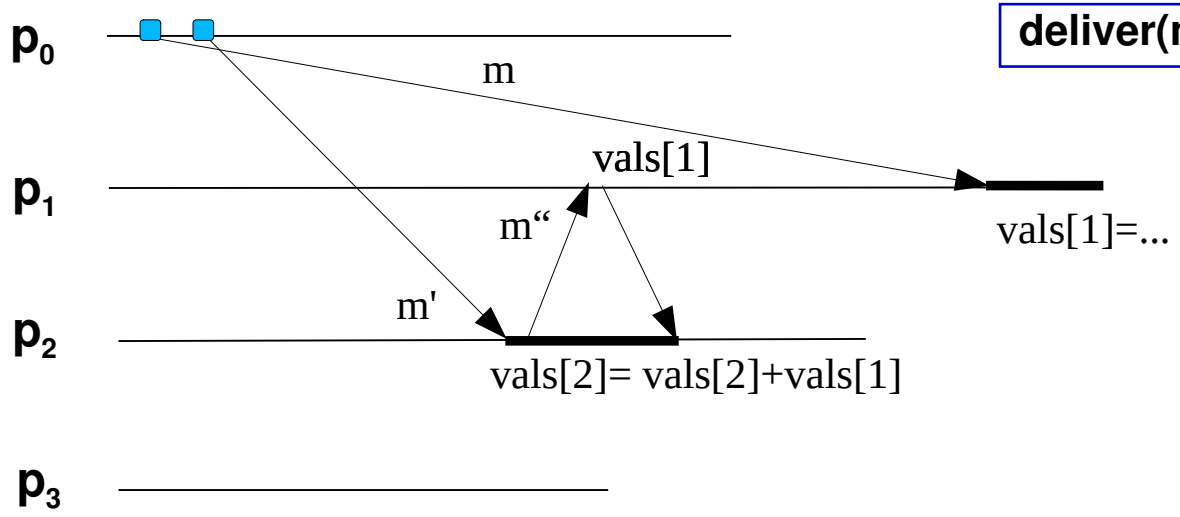
```
At P0:
  void init(Task tasks[]) {
    for (int i=1; i < tasks.length;i++)
      vals[i] = tasks[i].computeValue();
  }
```

We have at P0 :

send(m) → send(m') → send(m")

But we do not have :

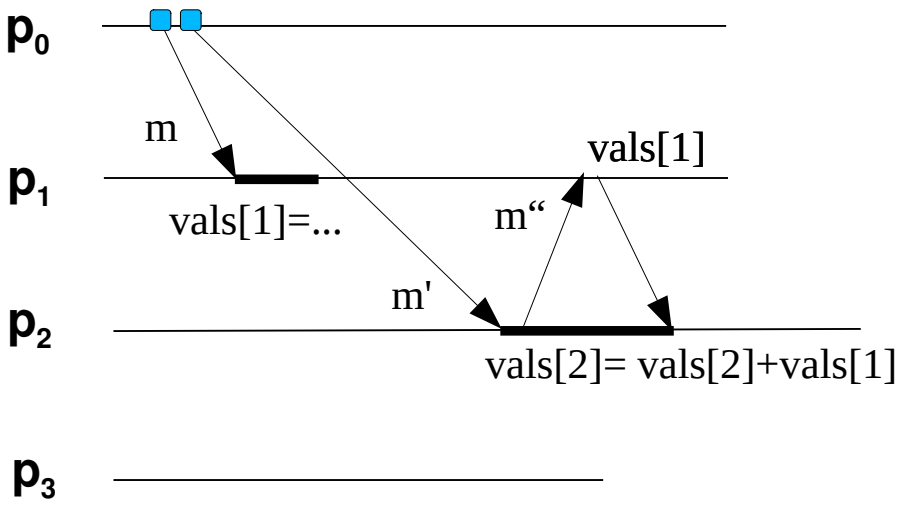
deliver(m) → deliver(m') → deliver(m")



Causality Example

```
At P0:
  void init(Task tasks[]) {
    for (int i=1; i < tasks.length;i++)
      vals[i] = tasks[i].computeValue();
  }
```

send (m) → send (m') → send (m")
⇒ deliver (m) → deliver (m') → deliver (m")



You must have point-to-point causality to be correct… But here, it would be only by chance that the execution is causal.

***Point-to-point causality is the developers‘ responsibility**, by choosing the proper design...*

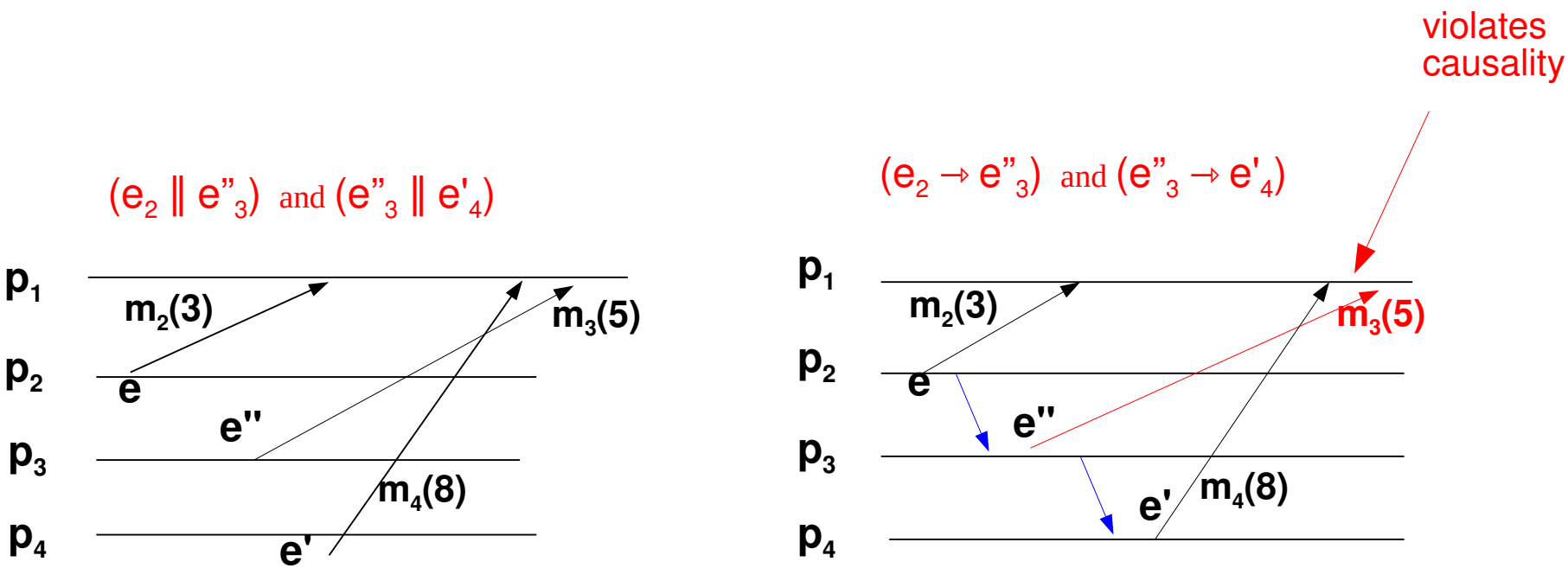
Point-to-Point Causality

- The Challenge of Point-to-Point Causality
  - When should P_1 deliver $\mathbf{m_4(8)}$?
  - Does it have to wait for $\mathbf{m_3(5)}$?

send (m) → send (m')
⇒ deliver (m) → deliver (m')

- How does a process detect missing or delayed events?
  - The two situations below are non-distinguishable from P_1's perspective
  - Enforcing the correct causal delivery is hard and very costly to do automatically

violates causality

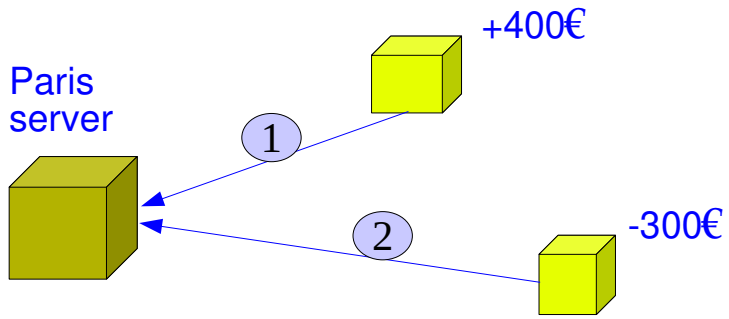


Discussing Total Ordering

- Example
  - Your bank uses a server in Paris
  - Every banking operation you do is done at the server
- You walk in your bank to deposit 400€ and withdraw 300€
  - A clerk deposits your money
  - But he asks that you use an ATM to withdraw

Is this the only possible execution?

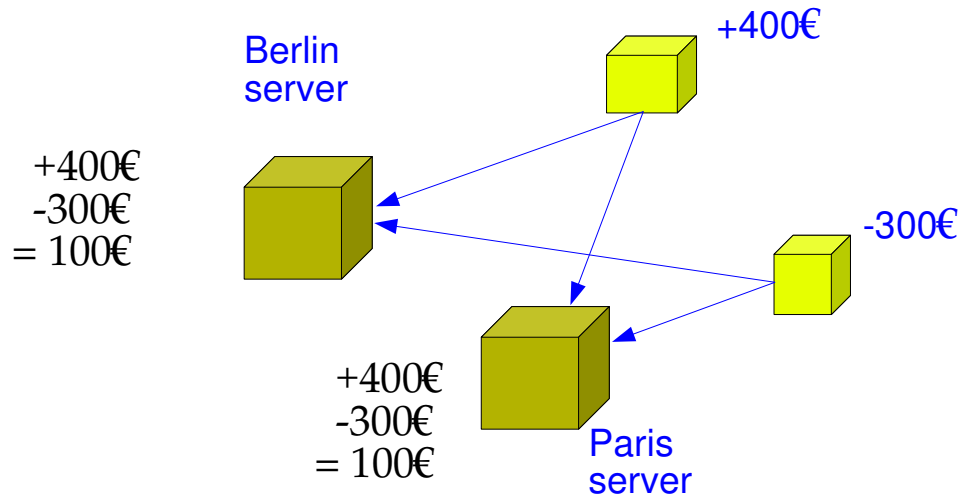
+400€
-300€
= 100€



Discussing Total Ordering

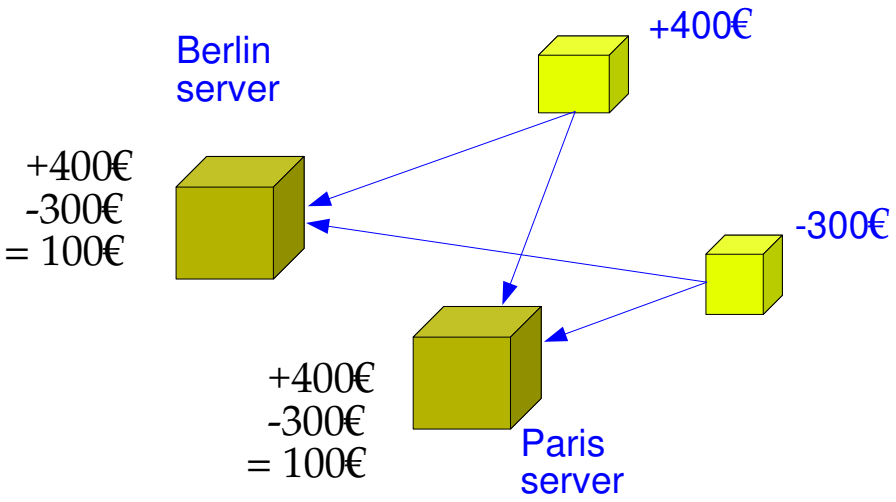
- Example
  - Your bank is now safer, it uses replicated computer systems in Paris and Berlin
  - Every banking operation you do is applied at both sites
- You walk in your bank to deposit 400€ and withdraw 300€
  - A clerk deposits your money
  - But he asks that you use an ATM to withdraw

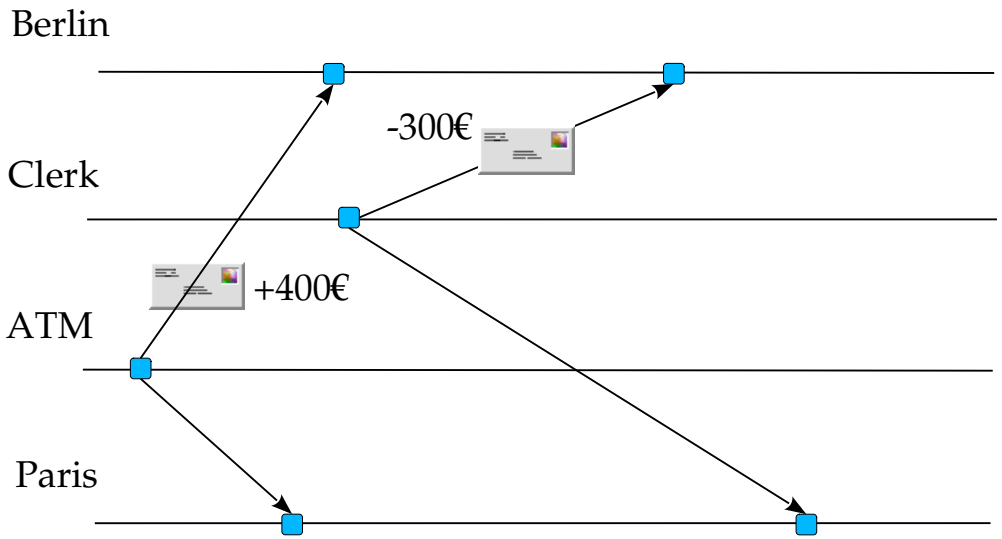
Is this the only possible execution?



Discussing Total Ordering

- Using UDP sockets?
  - Messages can be lost or re-ordered
- Using TCP sockets?
  - FIFO and lossless properties

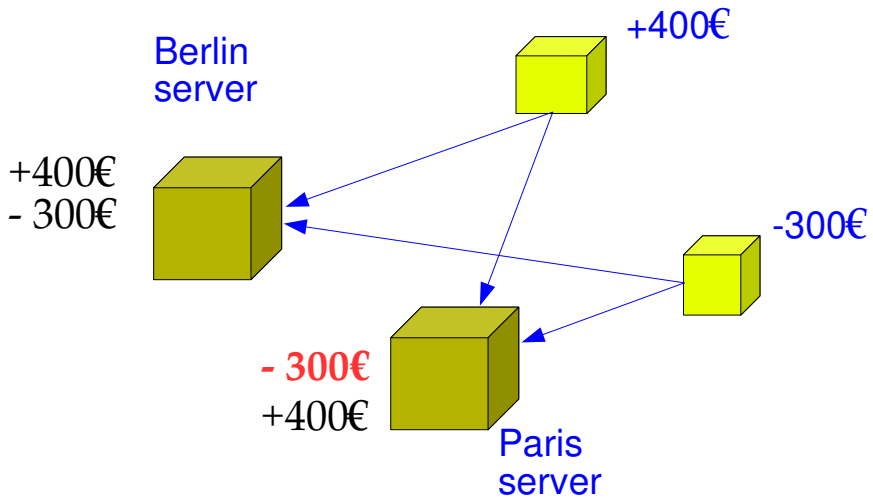


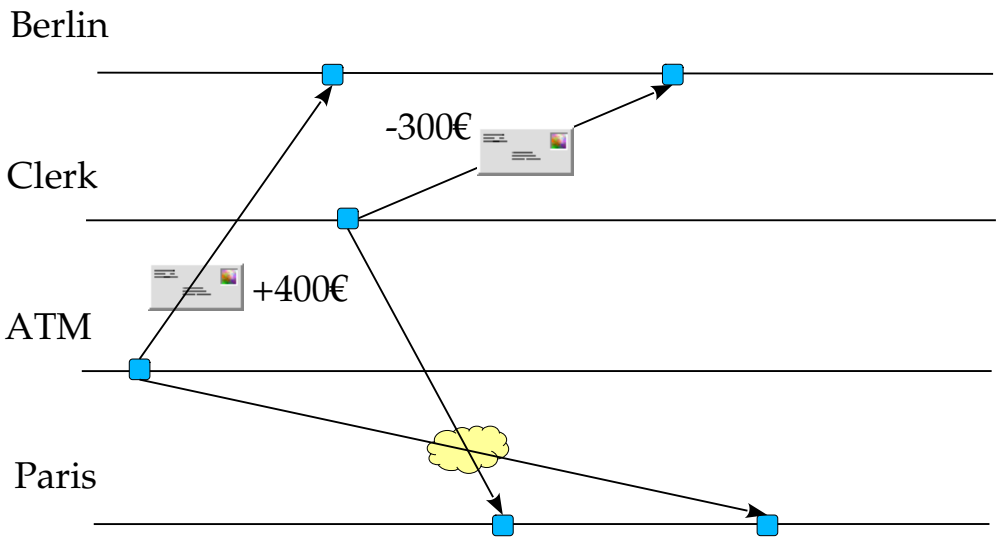


Is this the only possible execution?

Discussing Total Ordering

- Using TCP sockets
  - FIFO and lossless properties
  - But only point-to-point





We want the same order everywhere...

Totally Ordered Multicast

- Static group of N processes/actors
  - Members know each others
  - Each member has a communication channel with all the other members in the group
  - We assume FIFO and loss-less communication channels between all processes
- Totally-ordered Multicast
  - Any member may send any message at any time
  - Any sent message is sent to all members of the group
  - All received messages are delivered in the same order by all members

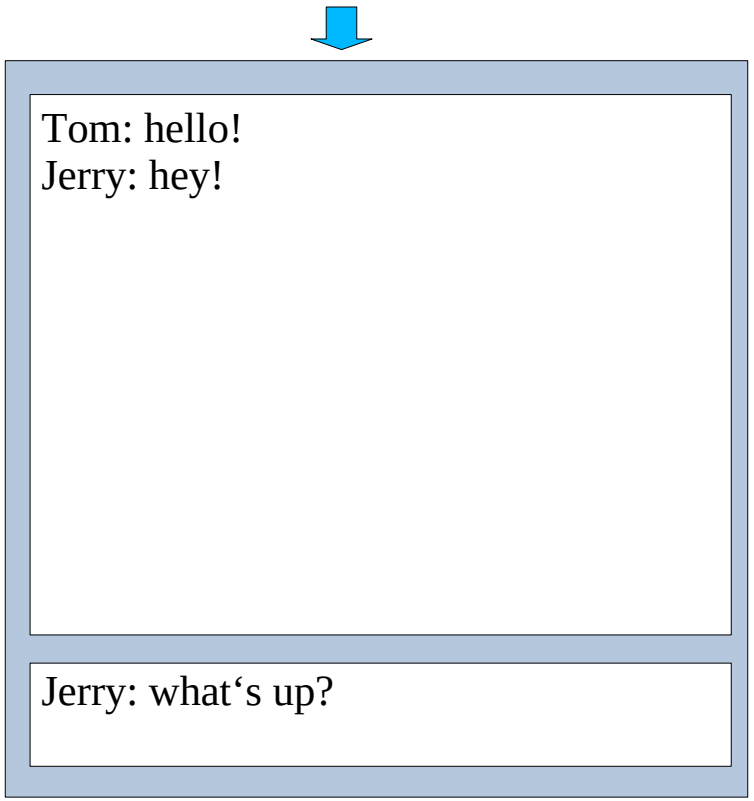
```
interface TotallyOrderedMulticast {
  interface Listener {
    void delivered(String message);
  }
  void set(Listener l);
  void multicast(String message);
}
```

Discussing Using a Totally Ordered Multicast

- Building an instant messaging application...

```
interface TotallyOrderedMulticast {
  interface Listener {
    void delivered(String message);
  }
  void set(Listener l);
  void multicast(String message);
}
```

Regular graphical interface,
like Swing or AWT in Java

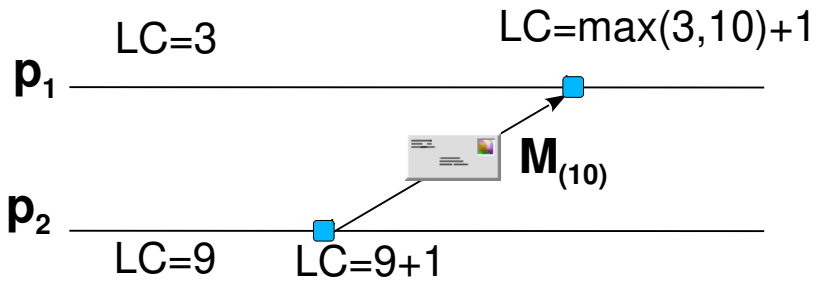


Q1) Received messages must be displayed when delivered. Can the code in the listener append the received string to the TextArea widget at the top of the window?

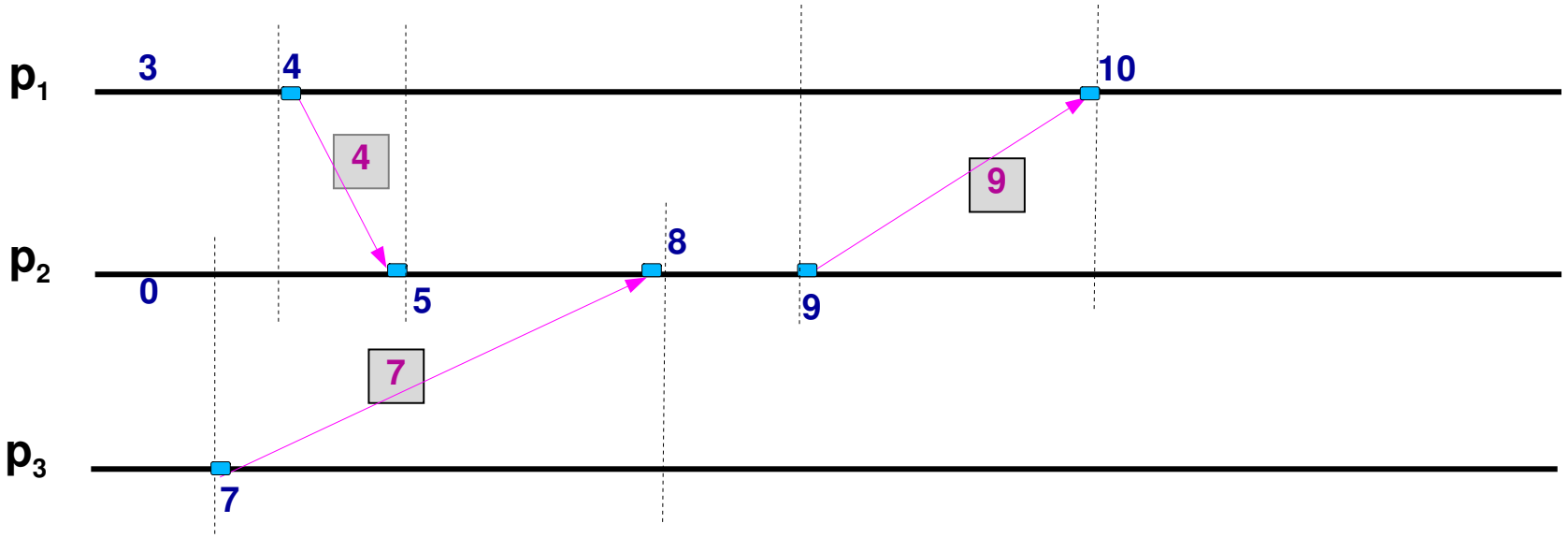
Q2) But what about messages typed in the bottom TextArea, sent by invoking the method '***multicast***', when should we append them to the TextArea widget at the top of the window ?

Logical Clocks (Lamport 1978)

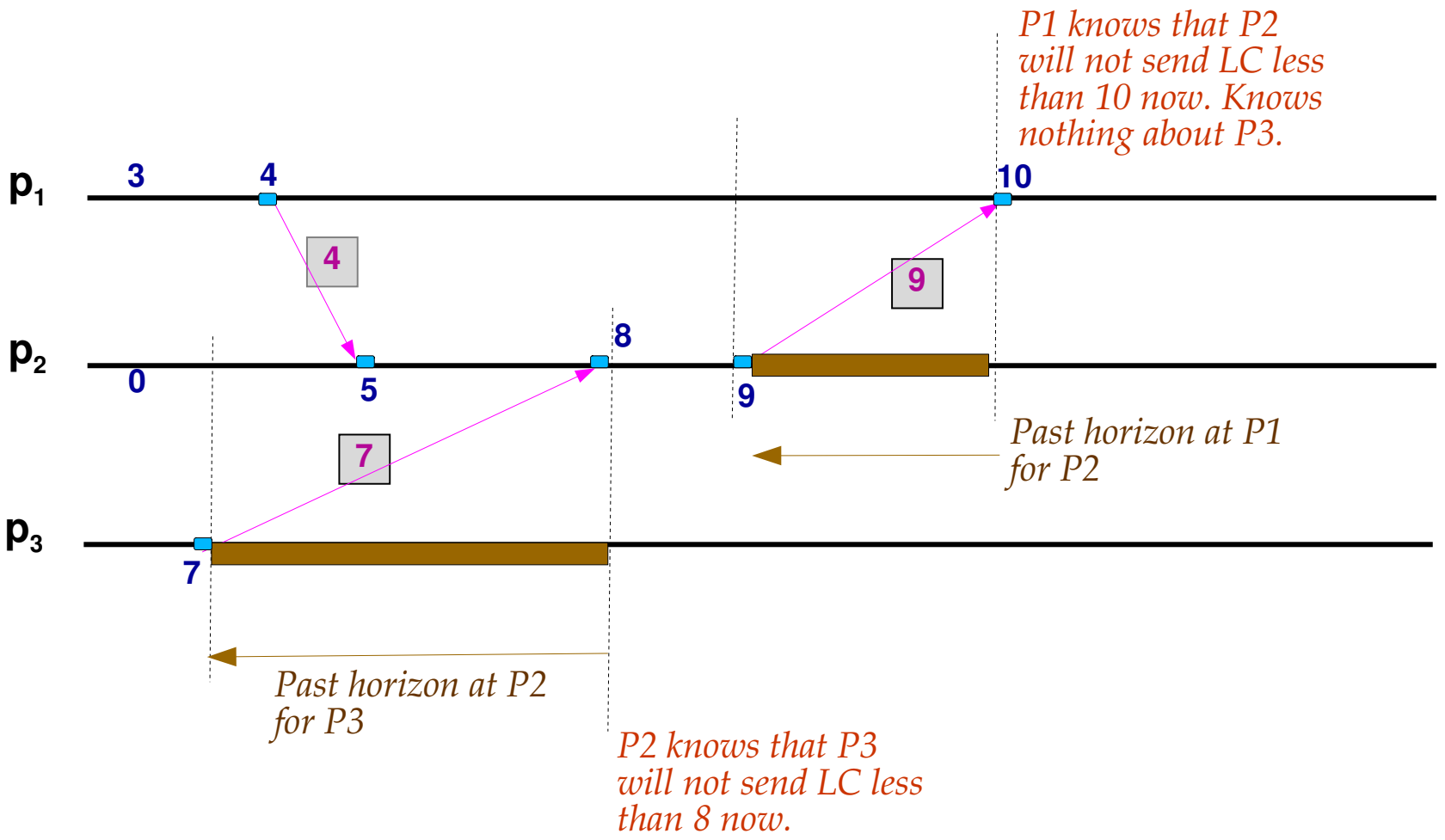
- Logical clocks capture message-based causality
  - Logical clocks are maintained as local counters
  - Logical clock for an event e_i^k is noted $\mathbf{LC(e_i^k)}$
  - For each new local event e_i^k: $LC(e_i^k) = LC(e_i^{k-1}) + 1$
- Sending a message M
  - This is a new local event: $\mathbf{LC(e_i^k) = LC(e_i^{k-1}) + 1}$
  - **M is timestamped with $LC(e_i^k)$**
- Receiving at P_j a message $M(LC(e_i^k))$
  - This is a new event e_j^r
  - $\mathbf{LC(e_j^r) = max(LC(e_j^{r-1}), LC(e_i^k)) + 1}$



Logical Clocks



Logical Clocks – Minimal Knowledge Propagation



Totally Ordered Multicast Using Lamport's Logical Clocks

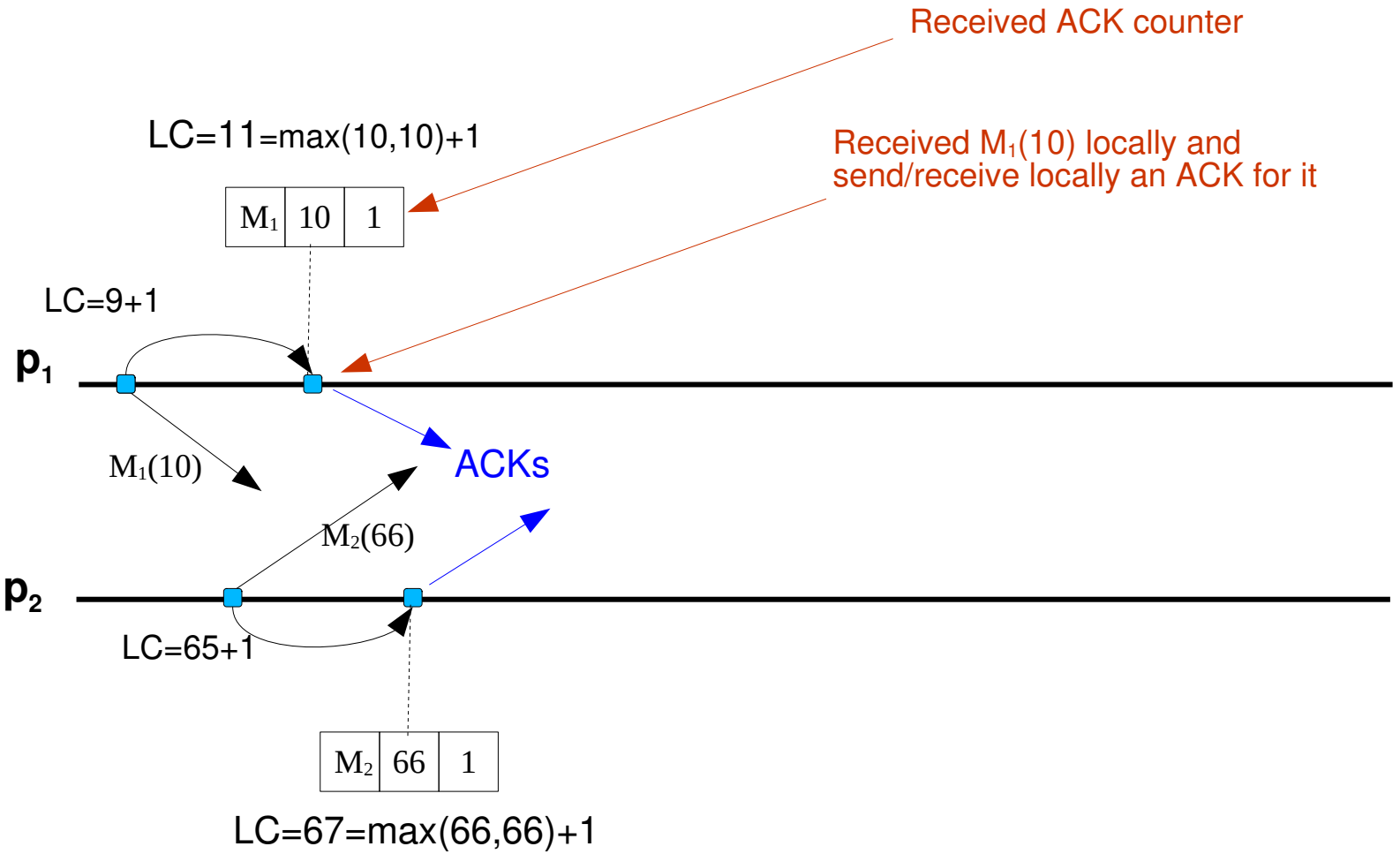
- Sending a message
  - The message is sent to the entire group
  - The message is ***timestamp***[(1)] with both the sender's logical clock and identifier
- Receive a message
  - Acknowledge each received message, by re-sending its timestamp to the entire group
  - Insert each received message in the local queue of received messages, keeping the queue ordered by increasing message timestamps
  - **Delivers** the first message in the queue of received messages (smallest timestamp), but only if it has been acknowledged by all other processes in the group

(1) Timestamp examples:
- clock 56, process 32 → timestamp=56.32
- clock 52, process 24 → timestamp=52.24

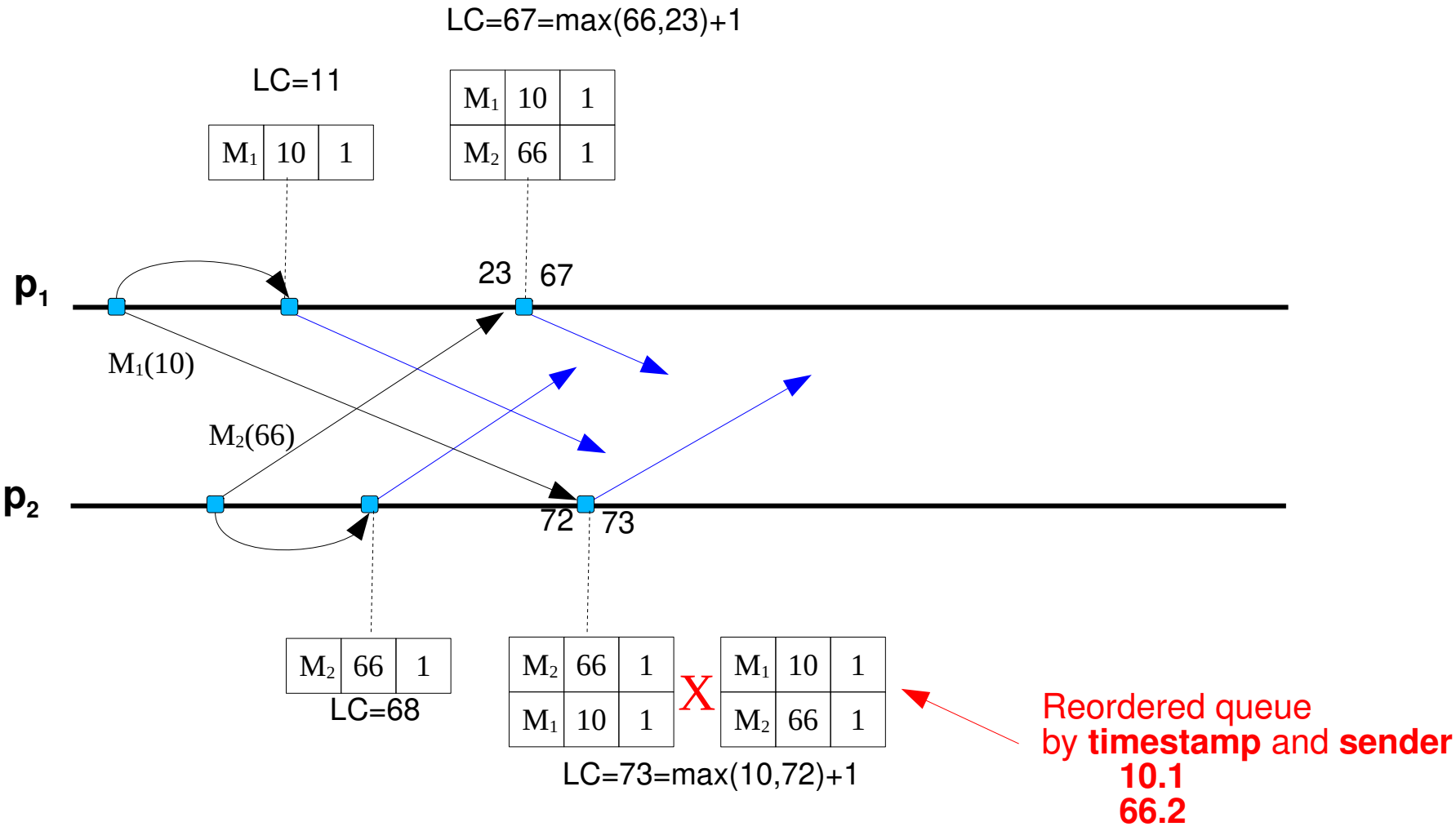
Classwork – Totally-Ordered Multicast

- First step – Static group
  - How do processes constitute a group?
- Second step – TOM – Static Group – No failures
  - Why does the protocol work? When a process P is about to deliver a message at the top of the queue of received messages, with all the necessary acknowledgments, why is it correct to do so? In other words, how can the process P be sure that it will not received any message in the future that would have a smaller timestamp that the one it is about to deliver?
- Third step – TOM – Process failures – No resurrection
  - Assume a perfect fault detector
  - Analyze the effect of process failures
  - Design a solution to resist process failures while maintaining the multicast atomicity
- Fourth step – TOM – Process failures & resurrections
  - Design a solution to resurrect failed processes

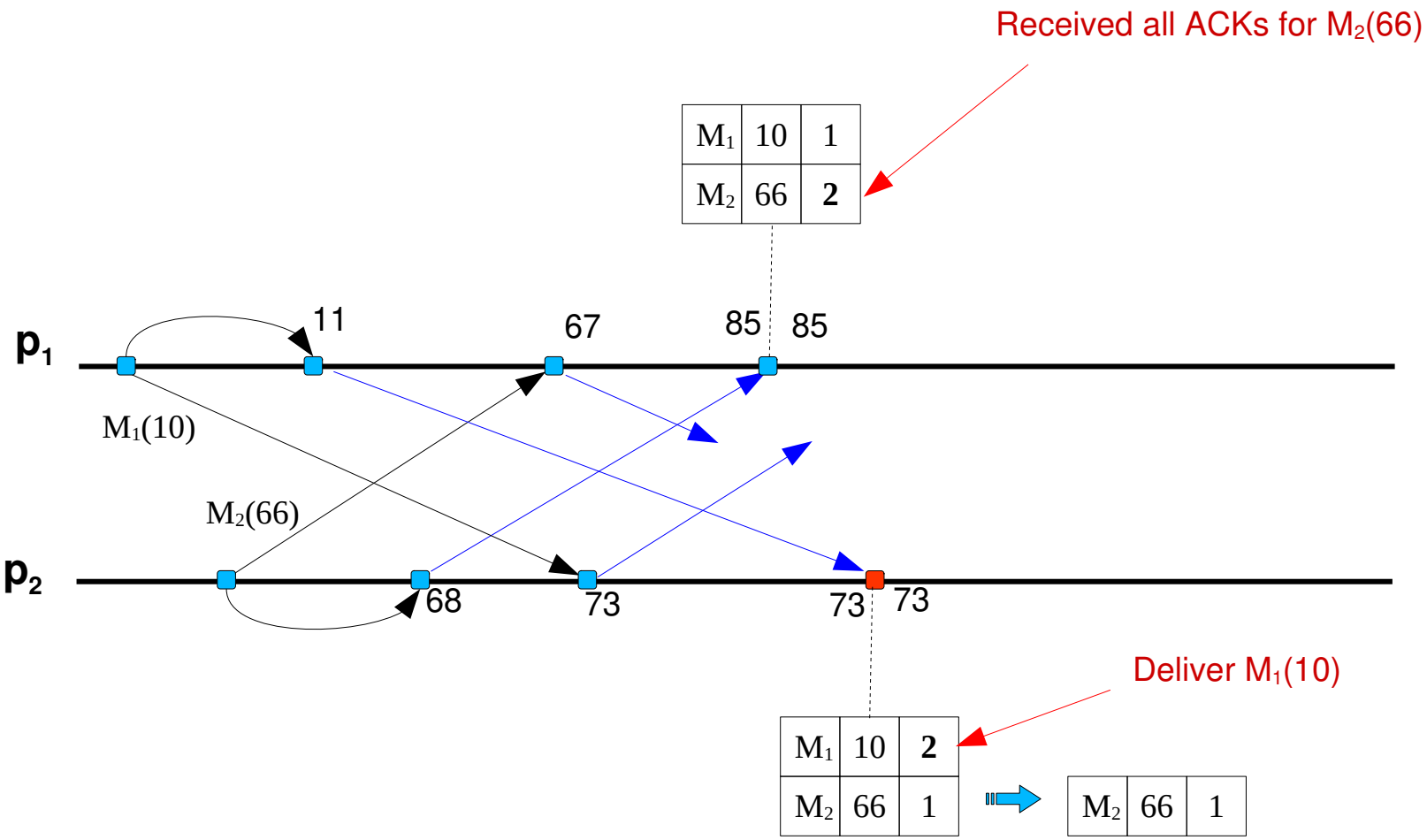
Totally Ordered Multicast



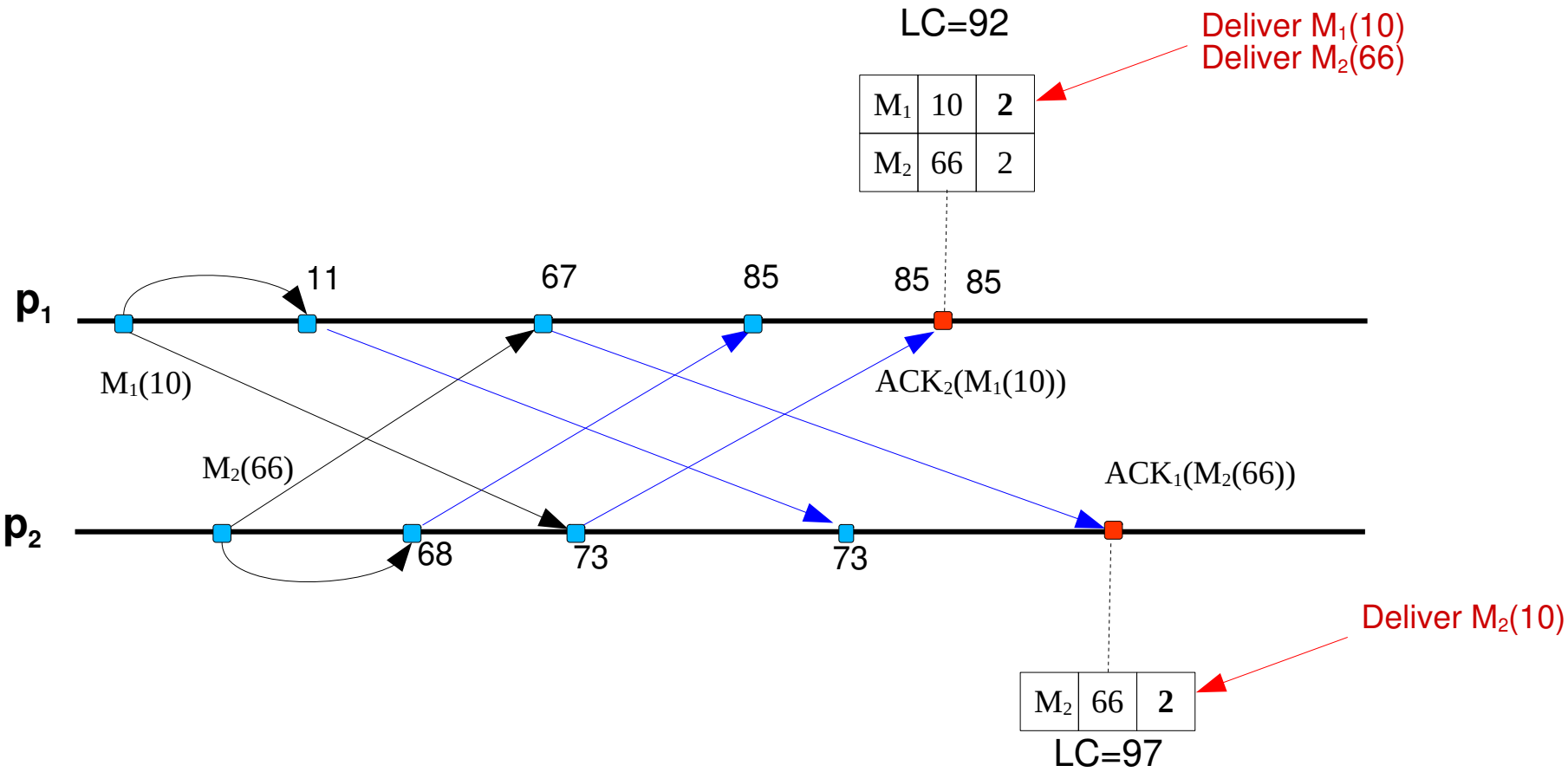
Totally Ordered Multicast



Totally Ordered Multicast



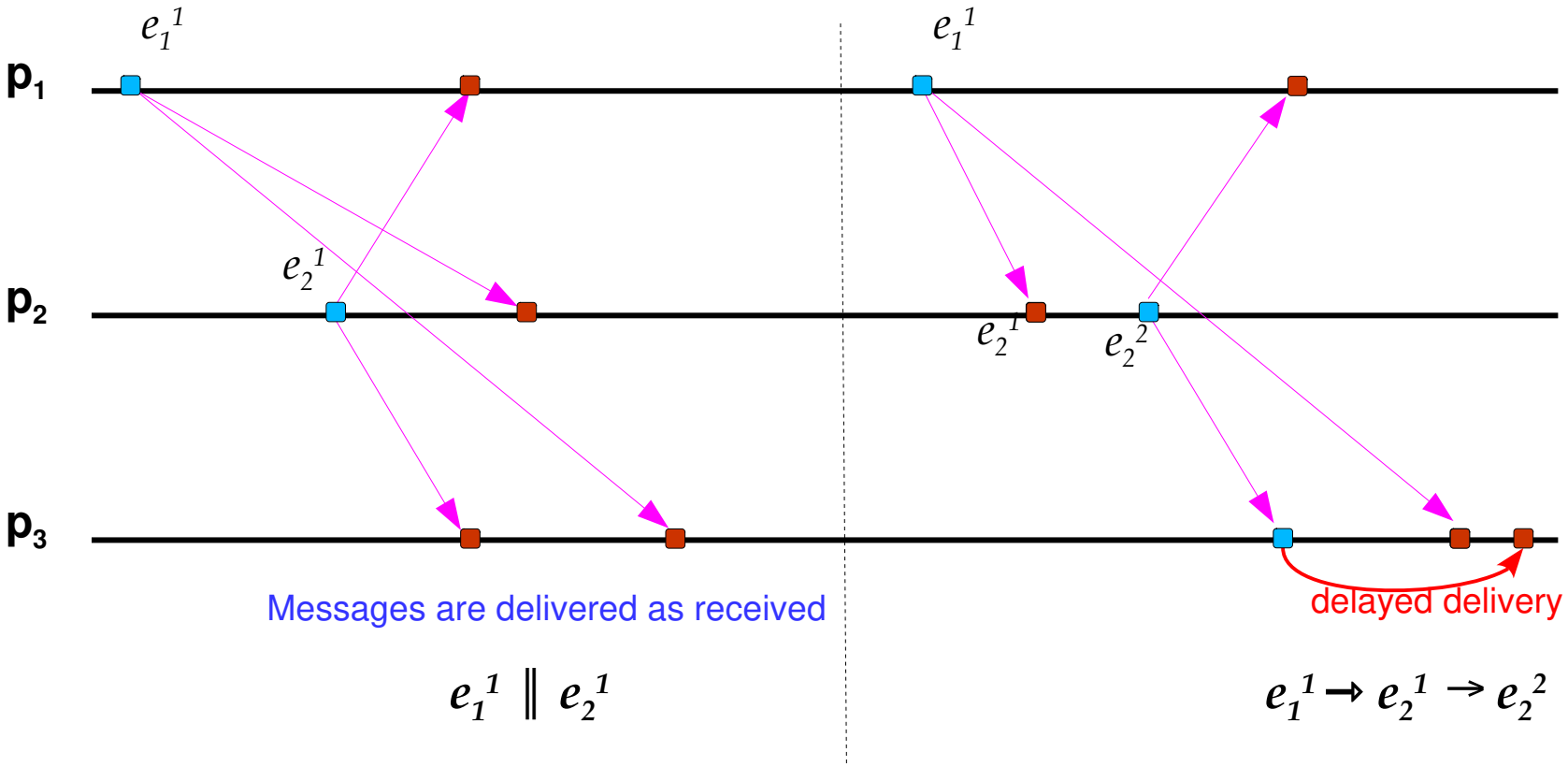
Totally Ordered Multicast



Totally-Ordered vs Causally-Ordered Multicast

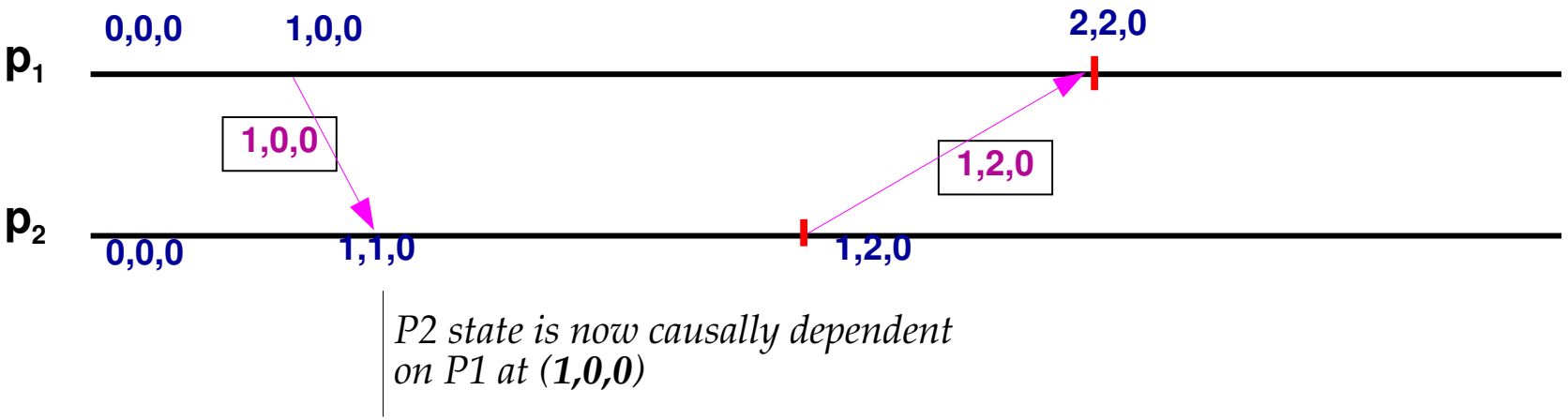
- Totally-ordered multicast
  - Everyone in the group sees all messages in the same order
- Causally-ordered multicast
  - Everyone in the group sees all messages in causal order
- The newsgroup example
  - Everyone sees a question first and the answers next
  - Answers may not be seen in the same order by everyone
  - Questions asked in parallel can be seen in different orders too

Causally-Ordered Multicast



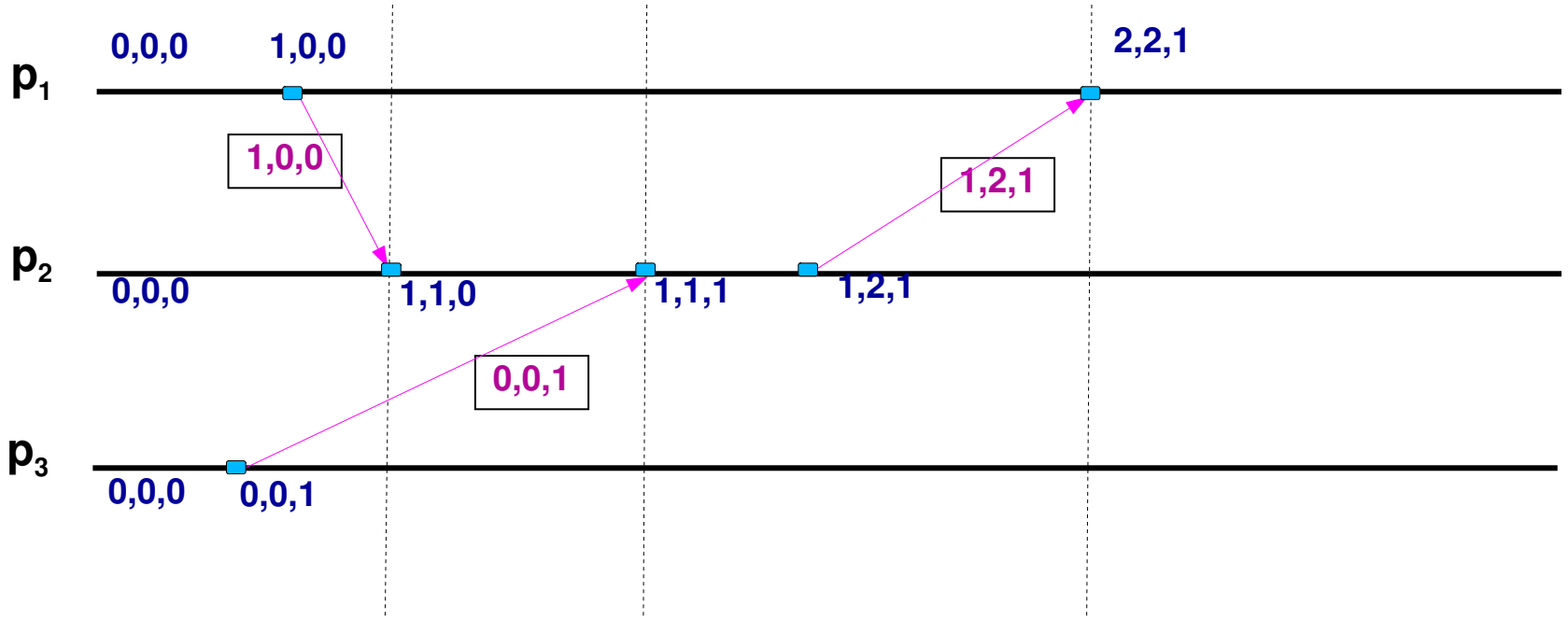
Vector Clocks – Definition

- Vector Clock (Fidge and Mattern, 1988)
  - A vector of logical clocks
    - One entry per known process P_i
    - VC[i] = max value of known LC(P_i)
  - Each event carries a vector clock
    - It records the processes' clocks that the event depends on
  - Each process P_i maintains a vector clock VC_i
    - Maintains the logical clocks that the current state of P_i depends on



Vector Clocks – Causality

- Causality & Vector Clock



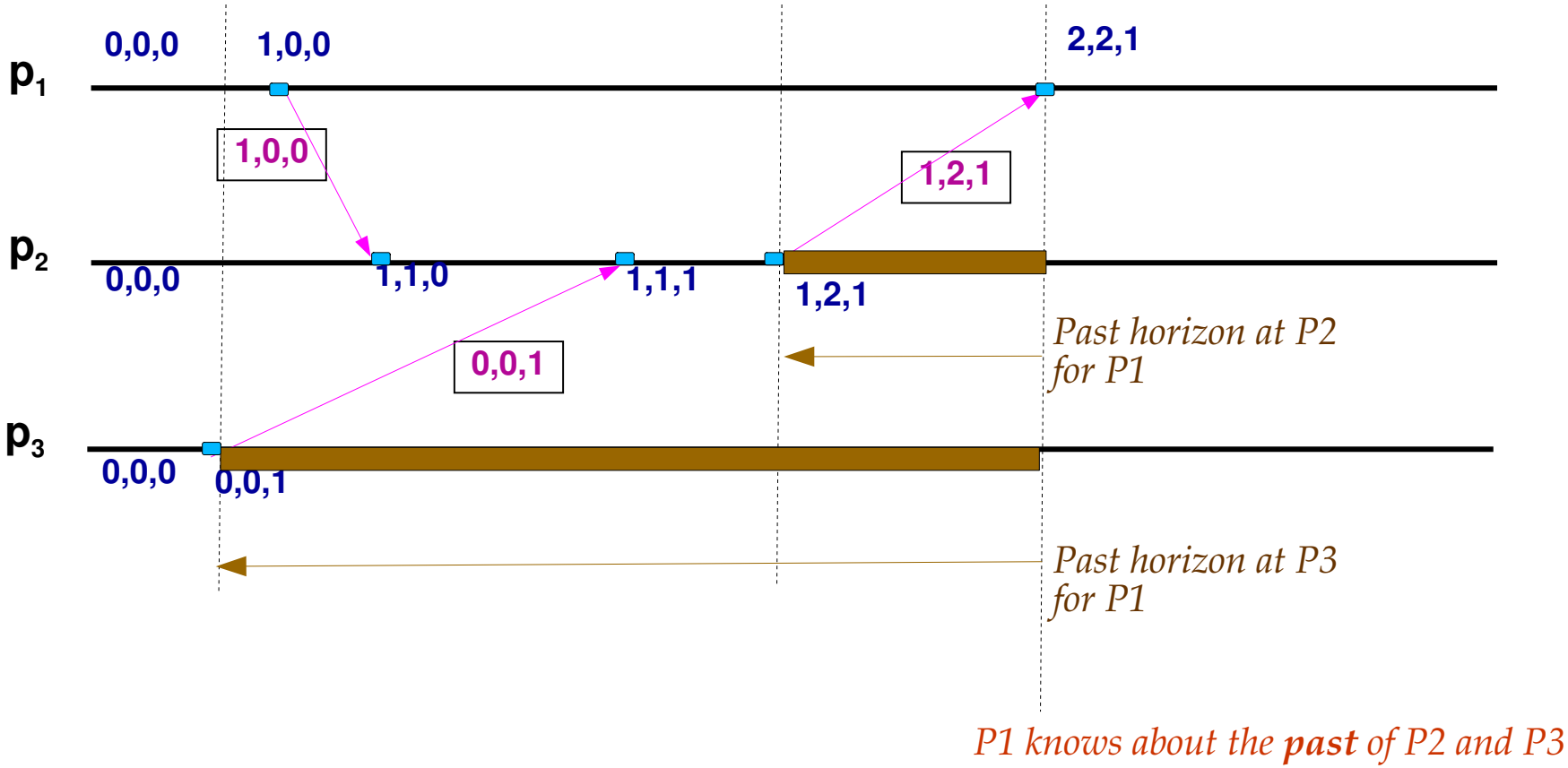
*P2 is now causally dependent on P1 at the time (**1,0,0**)*

*P2 is now causally dependent on P1 at the time (**1,0,0**) **and** on P3 at the time (**0,0,1**)*

*P1 is now causally dependent on P2 at the time (**1,2,1**) **and** on P3 at the time (**0,0,1**)*

Vector Clocks – Causality

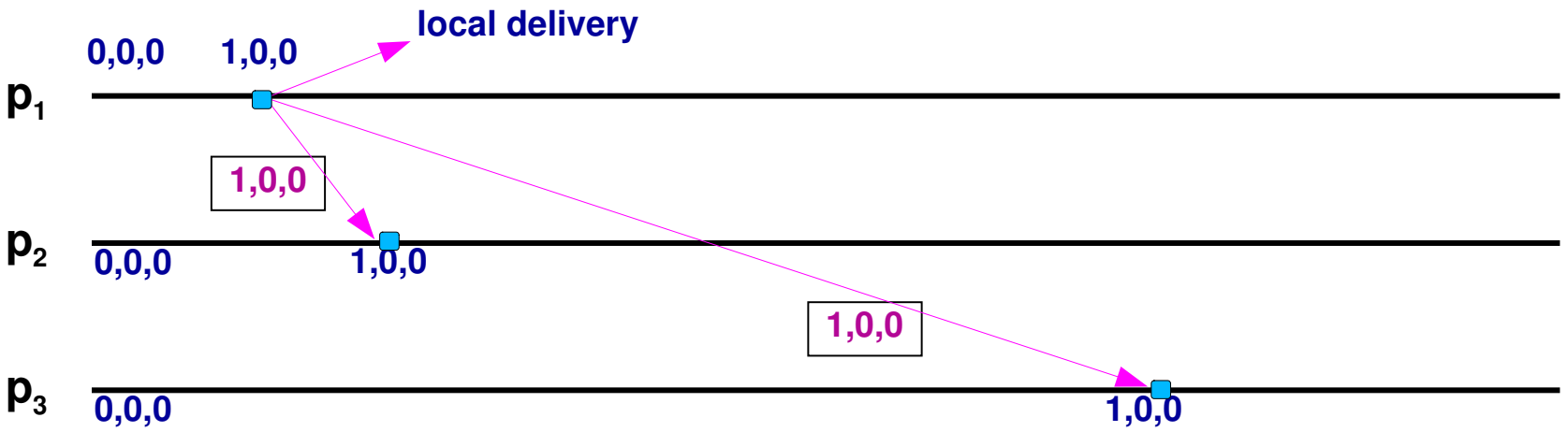
- Causality – A view in the past...



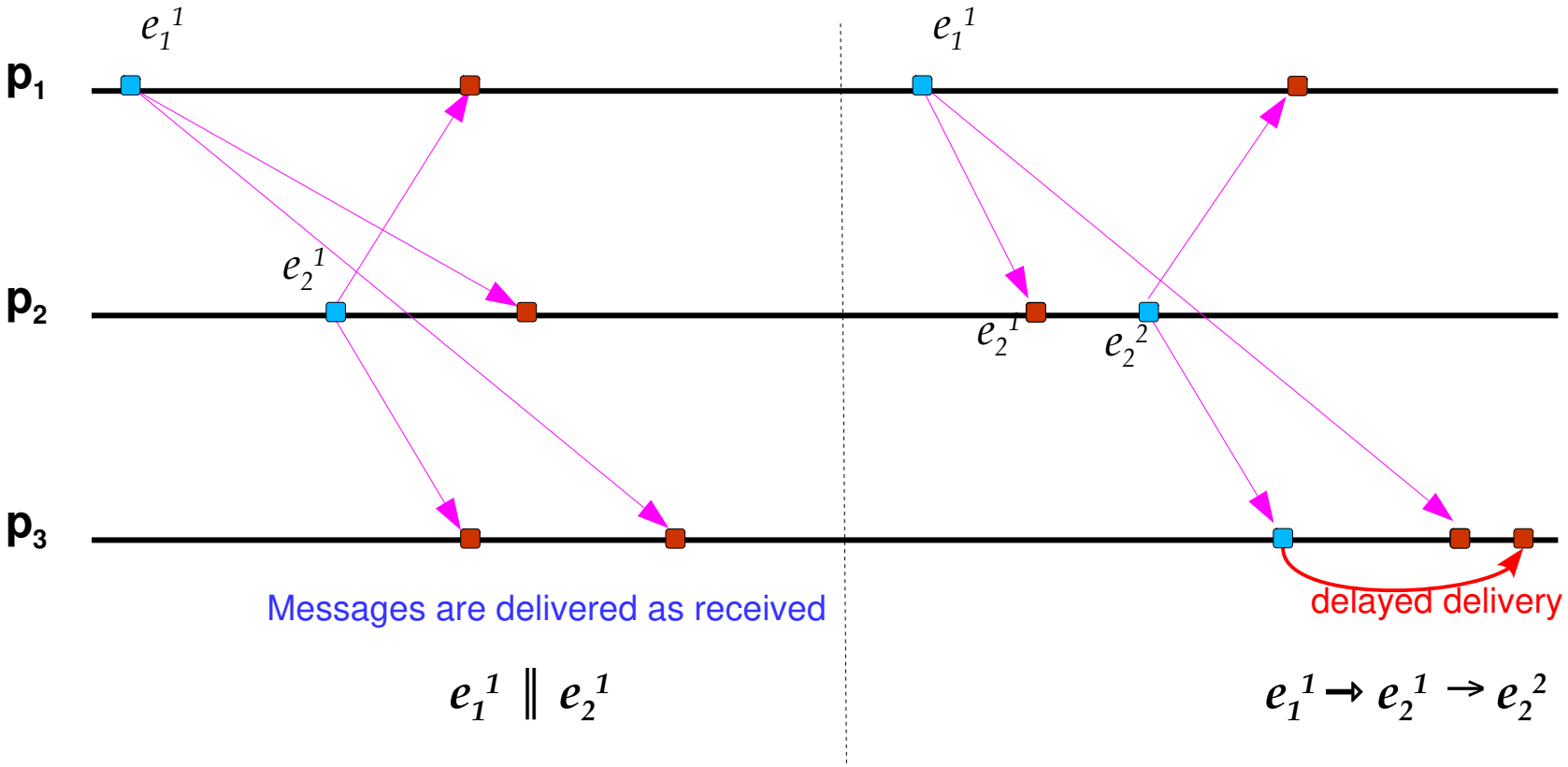
*P1 knows about the **past** of P2 and P3*

Causally-Ordered Multicast

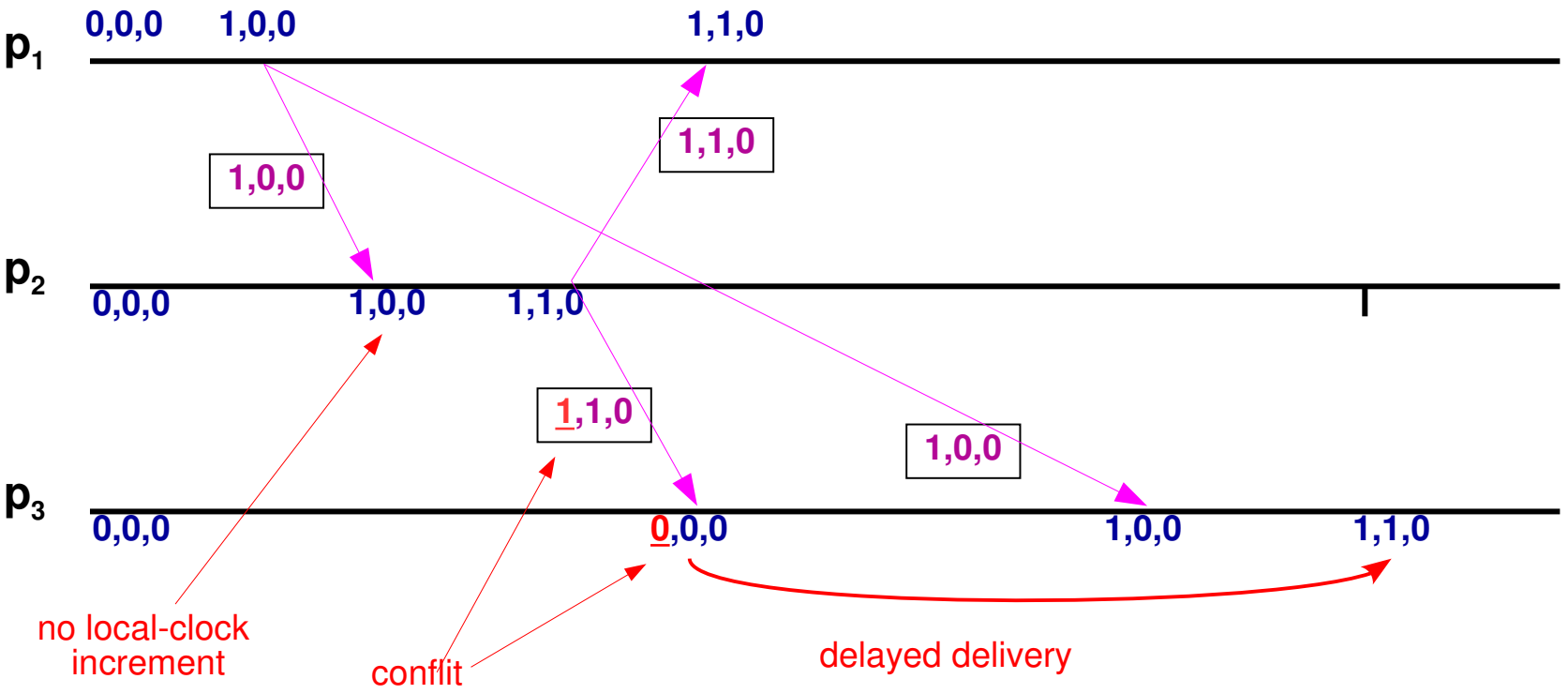
- Design
  - Each process maintains a vector clock per multicast group
  - Sending messages
    - **Increment local logical clock only regarding multicasting (no other events)**
    - Timestamp messages with its VC_i
  - Receiving messages
    - $VC_i[k] = \max(VC_i[i], VC[k])$ for all $k \neq i$
    - **No increment of local logical clock**



Causally-Ordered Multicast



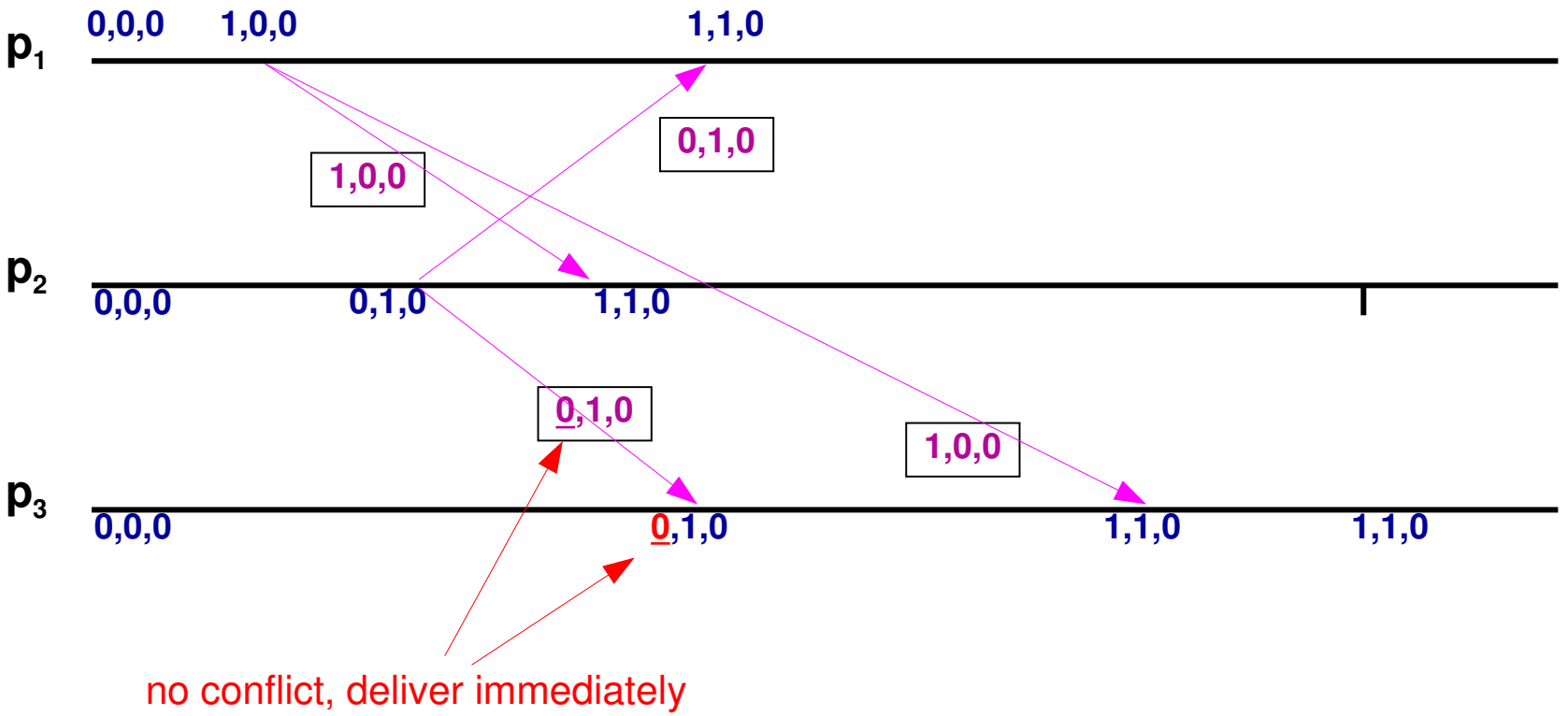
Causally-Ordered Multicast



For a message M, received by P_r from P_s, with vector clock VC_m
Delay delivery until

$VC_m[s] = VC_r[s]+1$
$VC_m[k] \leq VC_r[k]$ for all $k \neq s$

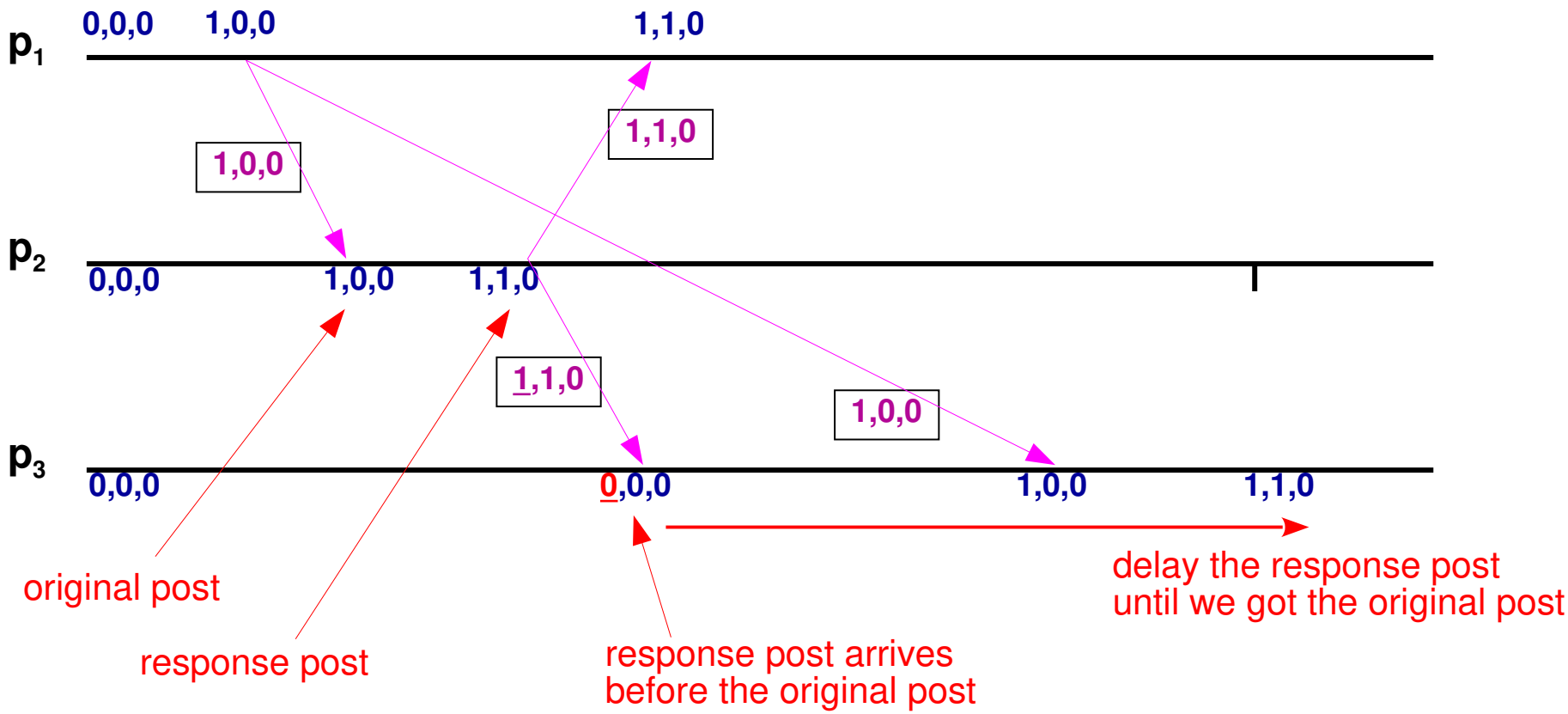
Causally-Ordered Multicast



Nota Bene: we avoided all the acknowledgment messages of the totally-ordered multicast

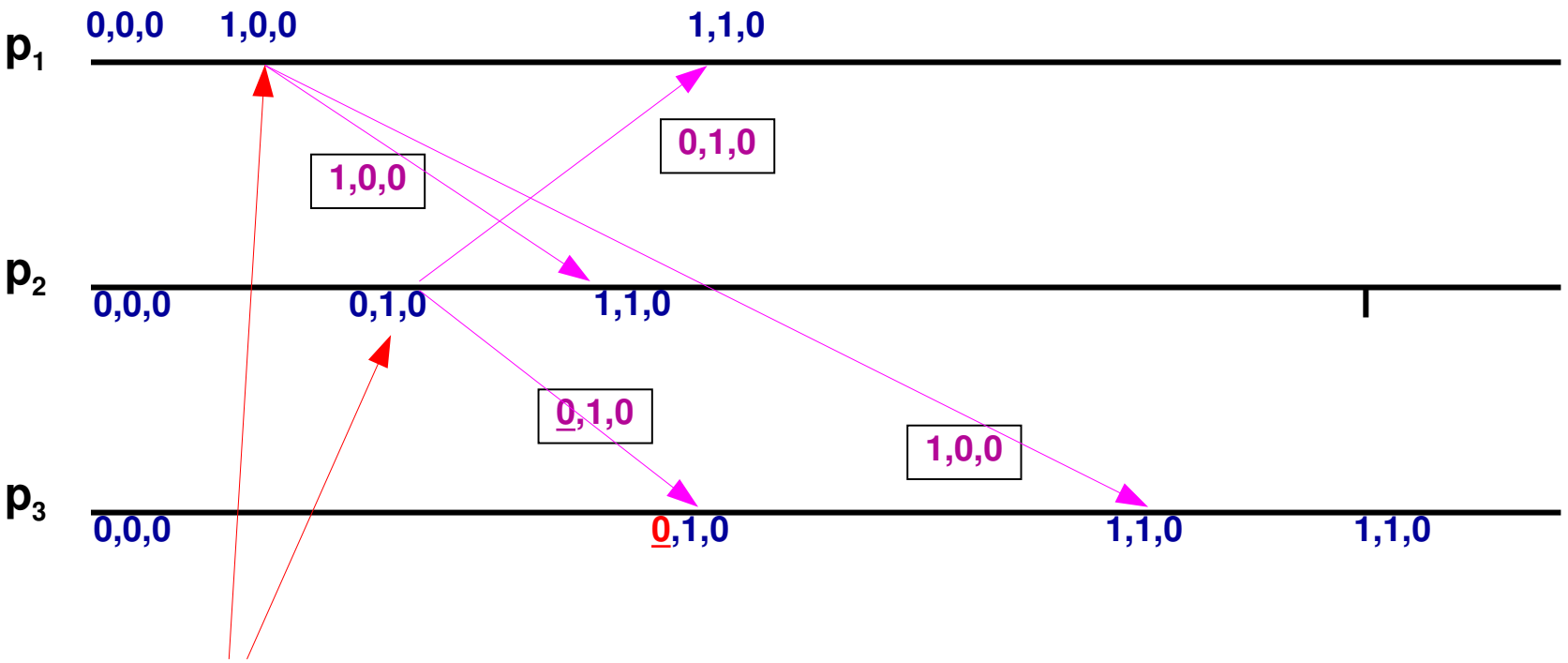
Causally-Ordered Multicast

- Example: newgroups
  - We want to avoid response posts to appear before the original posts



Causally-Ordered Multicast

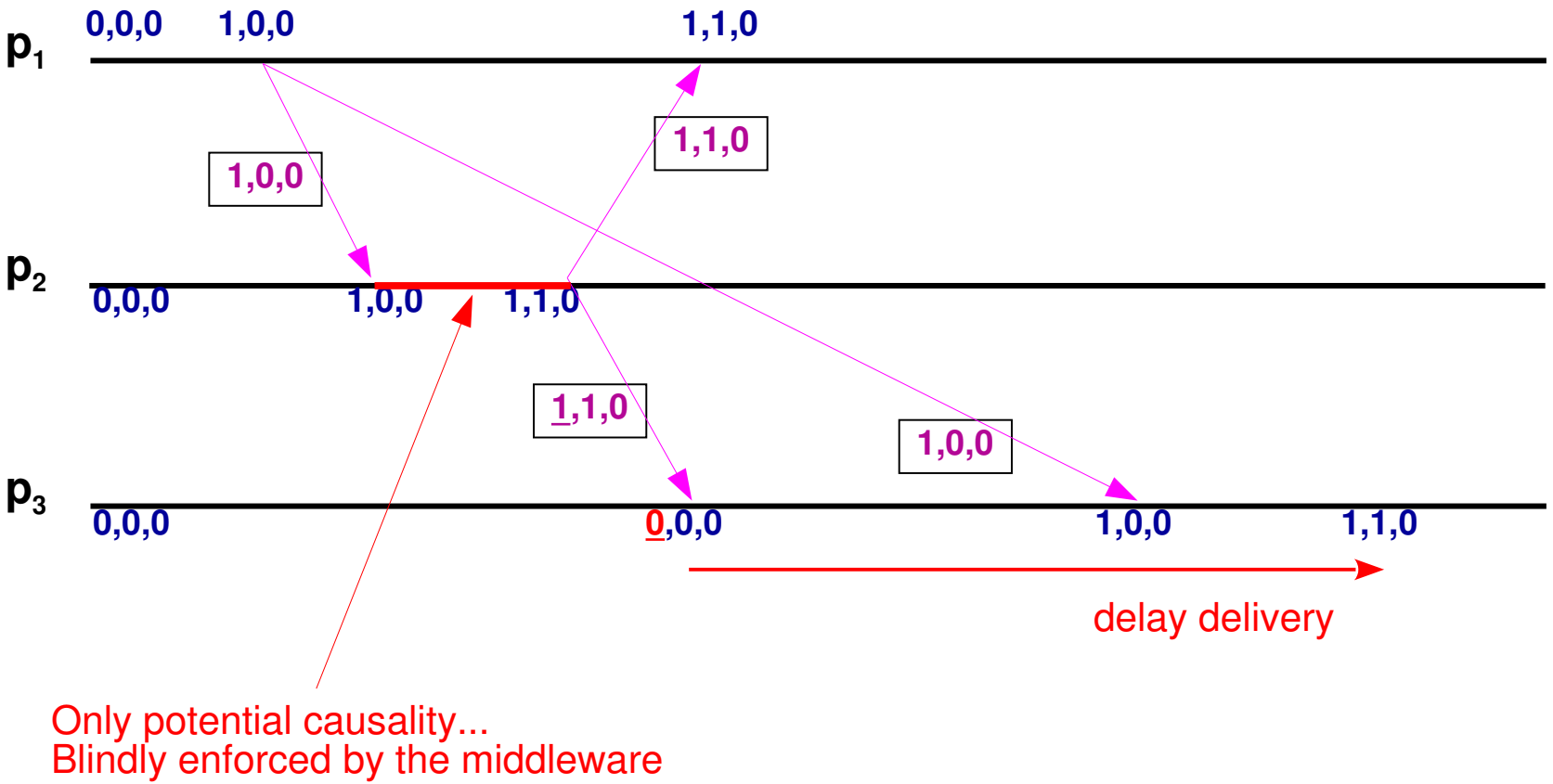
- Example: newsgroup
  - But we don't need to order concurrent posts...



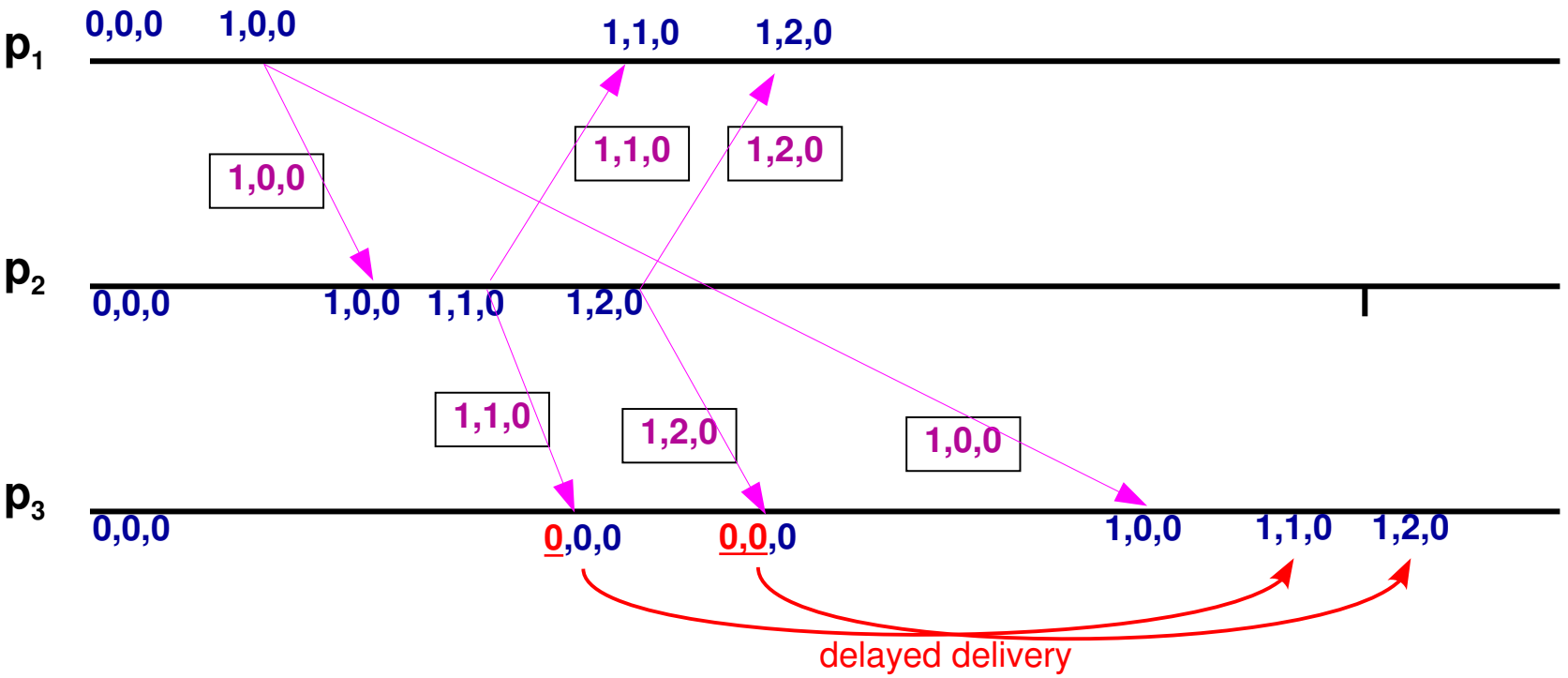
two independent posts, they don't have any order

Causally-Ordered Multicast

- Example - newsgroups
  - Notice that we don't know for a fact if the message is a response or original post
  - Our multicast library is blind to application-level semantics



Causally Ordered Multicast



For a message M

Received by P_r from P_s with vector clock VC

Delay delivery until

$VC_m[s] = VC_r[s]+1$

$VC_m[k] \leq VC_r[k]$ for all $k \neq s$

Classroom Study

- Handling process failures
  - How does the multicast survives the death of one or more processes?
- How can a process be resurrected?
  - Simple restart of the process on the same machine